ASSIGNMENT 6

Text Summarization Using BERT

Sr. No.	Name of Student	PRN	CO Mapped
1	Dhawse Spandan Yashwant	23070122085	CO5
2	Garvit Tyagi	23070122094	CO5

CA3 Project Title	RedactIQ: Explainable PII Detection and Redaction with Confidence-Scored Audit Trails

Aim

To implement a compact BERT encoder in PyTorch, pre-train it with the masked language modelling objective, and use its contextual sentence embeddings to produce an extractive summary of a RedactIQ audit report.

The experiment also demonstrates the property that distinguishes BERT from word2vec: the same word receives a different vector in different contexts.

Learning Outcome:

After completing this experiment, students will be able to:

- Implement a transformer encoder with multi-head self-attention, learned position embeddings and the special tokens used by BERT.
- Pre-train the encoder with the masked language modelling objective and interpret its predictions.
- Apply contextual sentence embeddings to extractive summarisation and distinguish extractive from abstractive approaches.

Theory

Recurrent models read a sentence one token at a time, so computation cannot be parallelised and distant words remain hard to relate. The transformer replaces recurrence with self-attention. Each token is projected into a query, a key and a value; the query is compared with every key to produce attention weights, and the output is the weighted sum of the values. Every token therefore attends to every other token in a single step, regardless of distance. Multi-head attention runs several such projections in parallel so that different heads can capture different relations, and because the mechanism is order-independent a position embedding is added to each token.

BERT, which stands for Bidirectional Encoder Representations from Transformers, is a stack of these encoder layers. It is encoder-only and bidirectional, so every token sees the whole sentence on both sides, which suits understanding tasks such as classification and entity recognition. GPT by contrast is decoder-only and strictly left to right, which suits generation. The input is wrapped between a CLS token, whose final state represents the whole sequence, and a SEP token that marks the end.

BERT is pre-trained with masked language modelling. About fifteen percent of the tokens are replaced by a MASK token and the model is trained to predict the original word from the surrounding context on both sides. The loss is computed only at the masked positions, so the label is set to an ignored value everywhere else. The original model was also trained with next sentence prediction, which later work

found contributes little. Pre-training requires no labelled data, which is why it can be run on very large corpora.

The consequence is that BERT embeddings are contextual. word2vec stores one vector for each word type, so the word name has an identical vector in every sentence in which it appears. BERT computes the vector of a token from its neighbours, so the same word in two different sentences receives two different vectors, and the cosine similarity between them is below one. This is the property measured in the experiment.

Summarisation is either extractive, which selects existing sentences, or abstractive, which generates new ones. Extractive summarisation is used here because it cannot introduce statements that are not in the source, which matters for a compliance document. Each sentence of the audit report is embedded by mean-pooling the final hidden states, the embeddings are grouped by k-means clustering, and the sentence closest to each cluster centre is selected. The selected sentences are then restored to their original order so that the summary reads in sequence.

The model used here is deliberately small, with two layers, four attention heads and ninety-six dimensions, and it is trained from scratch on the project corpus. The purpose of the experiment is the architecture and the pre-training objective, not model size; a published checkpoint would be used in a production system.

Algorithm

1. Generate a corpus of sentences of the kind that appear in a RedactIQ audit report.
2. Build a vocabulary including the special tokens for padding, CLS, SEP, MASK and unknown, and encode each sentence as the CLS token, the sentence tokens and the SEP token, padded to a fixed length.
3. Define the model: a token embedding, a learned position embedding, layer normalisation of their sum, a stack of transformer encoder layers with multi-head self-attention, and a linear masked language model head over the vocabulary.
4. Pass the padding mask to the encoder so that attention never reaches the padded positions.
5. Implement the masking function: select about fifteen percent of the real tokens, excluding padding, CLS and SEP, replace them by the MASK token, and set the label to the original token at those positions and to the ignored value elsewhere.
6. Pre-train with the Adam optimiser and a cross-entropy loss that ignores the value -100, skipping any batch in which no token was selected.
7. Evaluate the pre-training qualitatively by masking a token in a test sentence and printing the three highest-scoring predictions for that position.
8. Extract the vector of the same word from two different sentences and compute the cosine similarity between them, to show that the representation is contextual rather than static.
9. Embed each sentence of an audit report by mean-pooling its final hidden states over the non-padded positions.
10. Cluster the sentence embeddings with k-means, select the sentence nearest each centroid, restore the original ordering and report the resulting summary and the compression ratio.

Program

```python
import random
import warnings
import torch
import torch.nn as nn
from sklearn.cluster import KMeans
warnings.filterwarnings('ignore')

random.seed(42)
torch.manual_seed(42)

# ---------------- Step 1 : Corpus ----------------
# Sentences of the kind that appear in a RedactIQ audit report. A compact BERT
# is pre-trained on them from scratch, because the point of the experiment is
# the architecture and the masked-language-model objective, not model size.
SUBJ = ["the detector", "the regex rule", "the ner model", "the fusion layer",
        "the audit report", "the confidence score", "the redaction stage"]
VERB = ["identified", "flagged", "redacted", "reviewed", "scored", "matched"]
OBJ = ["an email address", "a phone number", "a pan number", "a person name",
       "an organisation name", "an aadhaar number"]
TAIL = ["with high confidence", "for manual review", "in the input document",
        "before translation", "during the audit"]

corpus = [f"{random.choice(SUBJ)} {random.choice(VERB)} {random.choice(OBJ)} "
          f"{random.choice(TAIL)}" for _ in range(4000)]
print("1. CORPUS :", len(corpus), "sentences")
print("   sample  :", corpus[0])
```

```python
# ---------------- Step 2 : Vocabulary with the special tokens ----------------
PAD, CLS, SEP, MASK, UNK = 0, 1, 2, 3, 4
vocab = {"[PAD]": PAD, "[CLS]": CLS, "[SEP]": SEP, "[MASK]": MASK, "[UNK]": UNK}
for s in corpus:
    for w in s.split():
        vocab.setdefault(w, len(vocab))
inv = {i: w for w, i in vocab.items()}
MAXLEN = 12
print(f"\n2. VOCABULARY : {len(vocab)} tokens including "
      f"[PAD] [CLS] [SEP] [MASK] [UNK]")

def encode(s):
    """Every sequence is wrapped as [CLS] ... [SEP], exactly as in BERT."""
    ids = [CLS] + [vocab.get(w, UNK) for w in s.split()][:MAXLEN - 2] + [SEP]
    return ids + [PAD] * (MAXLEN - len(ids))

X = torch.tensor([encode(s) for s in corpus])
print("   encoded :", [inv[i] for i in X[0].tolist()][:7], "...")

# ---------------- Step 3 : A compact BERT encoder ----------------
```

```python
class MiniBERT(nn.Module):
    """Token embeddings + learned position embeddings + a stack of transformer
    encoder layers. Each layer is bidirectional multi-head self-attention
    followed by a feed-forward network, which is the BERT architecture."""
    def __init__(self, nvocab, d=96, heads=4, layers=2):
        super().__init__()
        self.tok = nn.Embedding(nvocab, d, padding_idx=PAD)
        self.pos = nn.Embedding(MAXLEN, d)
        self.norm = nn.LayerNorm(d)
        layer = nn.TransformerEncoderLayer(d, heads, d * 4, dropout=0.1,
                                           batch_first=True, norm_first=True)
        self.enc = nn.TransformerEncoder(layer, layers)
        self.mlm = nn.Linear(d, nvocab)        # masked language model head

    def forward(self, x):
        p = torch.arange(x.size(1)).unsqueeze(0).expand_as(x)
        h = self.norm(self.tok(x) + self.pos(p))
        h = self.enc(h, src_key_padding_mask=(x == PAD))
        return h, self.mlm(h)

model = MiniBERT(len(vocab))
print(f"\n3. MODEL : compact BERT, 2 layers, 4 attention heads, 96 dimensions")
print(f"   parameters = {sum(p.numel() for p in model.parameters()):,}")

# ---------------- Step 4 : Masked language model pre-training ----------------
def mask_batch(x, p=0.15):
    """Replace 15 percent of real tokens by [MASK]. The labels are -100
    everywhere else, so the loss is computed only at the masked positions."""
    y = torch.full_like(x, -100)
    sel = (torch.rand_like(x, dtype=torch.float) < p) & \
          (x != PAD) & (x != CLS) & (x != SEP)
    y[sel] = x[sel]
    xm = x.clone()
    xm[sel] = MASK
    return xm, y

opt = torch.optim.Adam(model.parameters(), lr=3e-4)
lossf = nn.CrossEntropyLoss(ignore_index=-100)
print("\n4. MASKED LANGUAGE MODEL PRE-TRAINING")
for ep in range(1, 16):
    model.train()
    tot = n = 0.0
    for i in range(0, len(X), 64):
        xb = X[i:i + 64]
        xm, y = mask_batch(xb)
        if (y != -100).sum() == 0:             # no token was masked
            continue
        opt.zero_grad()
        _, logits = model(xm)
        loss = lossf(logits.reshape(-1, len(vocab)), y.reshape(-1))
        loss.backward()
        opt.step()
        tot += loss.item() * len(xb)
        n += len(xb)
    if ep in (1, 5, 10, 15):
        print(f"   epoch {ep:>2}  masked LM loss = {tot/n:.4f}")
```

```python
# ---------------- Step 5 : What the model learned ----------------
print("\n5. FILLING IN A MASKED TOKEN")
model.eval()
for sent, hide in [("the detector [MASK] an email address with high confidence", 2),
                   ("the regex rule matched a [MASK] number in the input document", 5)]:
    ids = [vocab.get(w, UNK) if w != "[MASK]" else MASK for w in sent.split()]
    ids = [CLS] + ids[:MAXLEN - 2] + [SEP]
    ids = ids + [PAD] * (MAXLEN - len(ids))
    with torch.no_grad():
        _, logits = model(torch.tensor([ids]))
    top = logits[0, ids.index(MASK)].topk(3).indices.tolist()
    print(f"   {sent}")
    print(f"      -> {', '.join(inv[i] for i in top)}")

# ---------------- Step 6 : Embeddings are contextual, not static ----------------
def embed(sentence):
    """Mean-pool the final hidden states over the real tokens."""
    ids = torch.tensor([encode(sentence)])
    with torch.no_grad():
        h, _ = model(ids)
    m = (ids != PAD).unsqueeze(-1).float()
    return (h * m).sum(1).squeeze(0) / m.sum()

def token_vec(sentence, word):
    ids = encode(sentence)
    with torch.no_grad():
        h, _ = model(torch.tensor([ids]))
    return h[0, ids.index(vocab[word])]

cos = nn.CosineSimilarity(dim=0)
a = token_vec("the detector flagged a person name for manual review", "name")
b = token_vec("the ner model scored an organisation name during the audit", "name")
print(f"\n6. CONTEXTUAL vs STATIC EMBEDDINGS")
print(f"   the word 'name' in two different sentences")
print(f"   cosine similarity of the two vectors = {cos(a, b):.3f}")
print("   word2vec would give exactly 1.000 here, because it stores one vector")
print("   per word type. BERT gives a different vector for each occurrence.")
```

```python
# ---------------- Step 7 : Extractive summarisation of an audit report ----------------
REPORT = [
    "the regex rule matched an email address with high confidence",
    "the detector identified a phone number in the input document",
    "the ner model flagged a person name for manual review",
    "the fusion layer scored an organisation name with high confidence",
    "the redaction stage redacted a pan number in the input document",
    "the confidence score reviewed an aadhaar number during the audit",
    "the audit report identified a person name before translation",
    "the detector matched an email address for manual review",
    "the ner model redacted an organisation name during the audit",
]
V = torch.stack([embed(s) for s in REPORT]).numpy()
K = 3
km = KMeans(n_clusters=K, random_state=42, n_init=10).fit(V)
picked = []
for c in range(K):
    d = ((V - km.cluster_centers_[c]) ** 2).sum(1)
    d[[i for i in range(len(REPORT)) if km.labels_[i] != c]] = 1e9
    picked.append(int(d.argmin()))
picked.sort()                                   # restore the original order

print(f"\n7. EXTRACTIVE SUMMARY OF A {len(REPORT)}-SENTENCE AUDIT REPORT")
print(f"   sentences are embedded with the compact BERT, grouped into {K}")
print(f"   clusters, and the sentence nearest each centroid is selected")
for c in range(K):
    members = [i + 1 for i in range(len(REPORT)) if km.labels_[i] == c]
    print(f"   cluster {c + 1} : sentences {members}")
print("\n   SUMMARY")
for i in picked:
    print(f"   [{i + 1}] {REPORT[i]}")
print(f"\n   compression : {len(REPORT)} sentences -> {len(picked)} "
      f"({100*len(picked)/len(REPORT):.0f} percent retained)")
```

Output

```
1. CORPUS : 4000 sentences
   sample  : the confidence score identified an email address in the input document

2. VOCABULARY : 47 tokens including [PAD] [CLS] [SEP] [MASK] [UNK]
   encoded : ['[CLS]', 'the', 'confidence', 'score', 'identified', 'an', 'email'] ...
```

```
3. MODEL : compact BERT, 2 layers, 4 attention heads, 96 dimensions
   parameters = 234,095

4. MASKED LANGUAGE MODEL PRE-TRAINING
   epoch  1  masked LM loss = 2.5697
   epoch  5  masked LM loss = 0.4429
   epoch 10  masked LM loss = 0.3679
   epoch 15  masked LM loss = 0.3519
```

```
5. FILLING IN A MASKED TOKEN
   the detector [MASK] an email address with high confidence
      -> redacted, identified, matched
   the regex rule matched a [MASK] number in the input document
      -> phone, pan, person
```

```
6. CONTEXTUAL vs STATIC EMBEDDINGS
   the word 'name' in two different sentences
   cosine similarity of the two vectors = 0.622
   word2vec would give exactly 1.000 here, because it stores one vector
   per word type. BERT gives a different vector for each occurrence.
```

```
7. EXTRACTIVE SUMMARY OF A 9-SENTENCE AUDIT REPORT
   sentences are embedded with the compact BERT, grouped into 3
   clusters, and the sentence nearest each centroid is selected
   cluster 1 : sentences [1, 4, 6, 7, 9]
   cluster 2 : sentences [3, 8]
   cluster 3 : sentences [2, 5]

   SUMMARY
   [1] the regex rule matched an email address with high confidence
   [3] the ner model flagged a person name for manual review
   [5] the redaction stage redacted a pan number in the input document

   compression : 9 sentences -> 3 (33 percent retained)
```

Explanation of the Output

The masked language modelling loss falls from 2.5697 to 0.3519 over fifteen epochs. It does not approach zero, and it should not: the objective is to predict a word that has been hidden, and several words are often equally plausible in the same slot, so a non-zero loss is the correct outcome rather than a sign of undertraining.

The fill-in-the-blank block shows what the model has learned. For the masked position in the detector MASK an email address, the three highest-scoring predictions are redacted, identified and matched, all of which are verbs that fit that slot. For a MASK number in the input document, the predictions are phone, pan and person, which are exactly the token types that precede the word number in the corpus.

The model has therefore learned the syntactic role and the semantic class of the hidden position from its context on both sides.

The contextual embedding measurement gives a cosine similarity of 0.622 for the word name taken from two different sentences. word2vec, which was trained in Assignment 3, would give exactly 1.000 for the same comparison, because it stores a single vector for each word type. This one number is the concrete difference between a static and a contextual embedding.

The summariser groups the nine audit sentences into three clusters and selects one representative from each. The three selected sentences describe a regex match on an e-mail address, an NER detection of a person name flagged for review, and a redaction of a PAN number, so each of the three detection routes of RedactIQ is represented exactly once and the near-duplicate sentences are suppressed. The report is compressed to thirty-three percent of its original length while the sentences are kept in their original order.

Conclusion

In this experiment a compact BERT encoder was implemented in PyTorch and pre-trained from scratch with the masked language modelling objective. The architecture comprised token and learned position embeddings, a stack of transformer encoder layers with bidirectional multi-head self-attention, and a masked language model head, with the sequence wrapped between the CLS and SEP tokens and the loss computed only at the masked positions. After fifteen epochs the model predicted plausible fillers for held-out masked positions, correctly recovering both the syntactic role and the semantic class of the hidden word from the context on either side. The cosine similarity of 0.622 between two occurrences of the same word in different sentences confirmed that the representation is contextual, in direct contrast to the word2vec embeddings of Assignment 3, which would have given a similarity of exactly one. The sentence embeddings were then used for extractive summarisation by k-means clustering, which reduced a nine-sentence audit report to three sentences that between them covered all three detection routes of RedactIQ. Extractive rather than abstractive summarisation was chosen because a compliance summary must not contain statements that are absent from the source. The model is deliberately small and is trained on a project-specific corpus, so the purpose of the experiment is the architecture and the pre-training objective rather than the accuracy of a production summariser; this completes the RedactIQ pipeline from preprocessing through detection, fusion, neural tagging, translation auditing and reporting.